SP ELEC 2A WEB APPLICATIONS DEVELOPMENT SPECIALIZATION · FINAL PROJECT

Build a PHP MVC Framework

You will design and implement a custom Model-View-Controller framework in PHP from scratch — applying PSR-4 autoloading, SOLID design principles, and delivering a working MVP application on top of your framework.

Type **Individual**	Weight **40% of grade**	Duration **3 weeks**	Language **PHP 8.3+**
Total Marks **100 pts**	Submission **GitHub**	Min. Pass **60 pts**	Academic Year **2025–2026**

1. Project Overview

> You are tasked with building a lightweight, production-quality MVC framework in PHP — similar in spirit to Laravel or Symfony, but written entirely by you. On top of that framework, you will deliver a working MVP application that demonstrates the framework's capabilities to a hypothetical client.

The project has two distinct layers:

- **Framework layer:** The framework layer — the engine you design and build.
- **Application layer:** The application layer — the MVP that runs on top of it.

Both layers are assessed independently. A broken MVP does not invalidate a well-built framework, and vice versa.

2. Technical Specifications

The following five requirements define the technical scope of your submission.

#	Requirement	Category	Detail
1	**PHP as Implementation Language**	*Language*	Written entirely in PHP 8.3+. Use modern features: union types, enums, named arguments, readonly properties, and match expressions. No PHP version below 8.3.
2	**MVC Framework Structure**	*Architecture*	Implement Model, View, and Controller layers with a front-controller pattern (public/index.php). Router resolves URIs to controller actions and supports route parameters such as /posts/{id}.
3	**PSR-4 Autoloading**	*Standards*	Define at least two PSR-4 namespaces in composer.json: Core\ for the framework and App\ for the

#	Requirement	Category	Detail
			application. No manual require or include for class files except the entry point.
4	**SOLID Design Principles**	*Design*	Apply all five SOLID principles visibly in your framework design. Provide a written justification document. A DI container is required to demonstrate Dependency Inversion.
5	**MVP Application**	*Deliverable*	A working CRUD application with at least 5 routes, form handling, input validation, and database interaction. Must prove the framework is usable in a real development context.

3. SOLID Design Principles

All five SOLID principles must be demonstrably present in your framework architecture. The table below maps each principle to its expected application. A written justification document (1–2 pages) cross-referencing specific classes and methods is a required deliverable.

	Principle	Core Rule	Applied in your framework
S	**Single Responsibility**	*One class, one reason to change.*	Router only routes; Request only wraps HTTP input; Response only wraps output. A controller must not build SQL queries. A model must not render HTML views.
O	**Open / Closed**	*Open for extension, closed for modification.*	Add a MySQLDriver or SQLiteDriver without touching Connection.php. Define a DatabaseDriver interface; new drivers implement it; existing code never changes.
L	**Liskov Substitution**	*Subtypes must be fully substitutable.*	MySQLDriver and SQLiteDriver must be interchangeable wherever a DatabaseDriver is expected — same method signatures, same contracts, no surprising side effects.
I	**Interface Segregation**	*No client forced to depend on unused methods.*	Avoid fat interfaces. Split Model into Findable and Persistable. A read-only repository implements only Findable; it is never forced to stub save() or delete().
D	**Dependency Inversion**	*Depend on abstractions, not concretions.*	Controllers receive PostRepositoryInterface, not MySQLPostRepository. A DI container binds the concrete class at runtime. High-level modules stay decoupled from infrastructure.

Code Examples

S — Single Responsibility (SRP)

Each class has one job. The Router only resolves routes; the Dispatcher only invokes controllers.

```
// Good: Router only routes
class Router {
    public function register(string $method, string $uri, array $action): void;
    public function resolve(Request $request): array;
}

// Bad: Router also dispatches and renders (two reasons to change)
class Router {
    public function handle(Request $req): Response { /* too much */ }
}
```

O — Open / Closed (OCP)

Extend behaviour through new code, not by modifying existing classes.

```
interface DatabaseDriver {
    public function connect(array $config): PDO;
}

// Add new drivers without touching Connection.php
class MySQLDriver  implements DatabaseDriver { ... }
class SQLiteDriver implements DatabaseDriver { ... }
```

D — Dependency Inversion (DIP)

Controllers depend on interfaces, not concrete classes. A DI container wires dependencies at runtime.

```
class PostController {
    public function __construct(
        private PostRepositoryInterface $posts,  // abstraction
        private ViewEngine $view
    ) {}
}

// Container binding in bootstrap
$app->bind(PostRepositoryInterface::class, MySQLPostRepository::class);
```

4. Expected Project Structure

Your repository must follow this structure. The namespace hierarchy must mirror the directory structure exactly for PSR-4 compliance.

```
my-mvc-framework/
|-- app/                  # Application layer (MVP)
|   |-- Controllers/
|   |-- Models/
|   |-- Views/
|   `-- Middleware/
|-- core/                 # Framework layer
|   |-- Http/
|   |   |-- Request.php
|   |   |-- Response.php
|   |   `-- Router.php
```

```
|   |-- Database/
|   |   |-- Connection.php
|   |   |-- QueryBuilder.php
|   |   `-- Model.php
|   |-- View/
|   |   `-- Engine.php
|   |-- Container/
|   |   `-- Container.php     # DI container
|   `-- Application.php
|-- config/
|   |-- app.php
|   `-- database.php
|-- public/               # Web server document root
|   `-- index.php         # Front controller (only file with require)
|-- routes/
|   `-- web.php
|-- composer.json         # PSR-4 autoload configuration
`-- README.md
```

Minimum PSR-4 section in composer.json:

```
"autoload": {
    "psr-4": {
        "Core\\": "core/",
        "App\\":  "app/"
    }
}
```

5. Deliverables

Submit all of the following by the stated deadline:

1. Source code — Full repository pushed to a private GitHub repo. The evaluator will be added as a collaborator before the deadline.
2. README.md — Setup instructions, framework design decisions, a list of all routes, and a description of the MVP application.
3. composer.json — Must define PSR-4 autoloading namespaces for both the framework core (Core\) and the application (App\).
4. MVP application — A working CRUD application demonstrating: at least 5 distinct routes, controller dispatch, database interaction via your model layer, view rendering, and form handling with input validation.
5. Design justification (1–2 pages) — A written reflection explaining how each SOLID principle is applied, with references to specific classes and methods in your codebase.

6. Grading Rubric

Each criterion is assessed on a 0–full scale. Partial credit is awarded for partial implementations.

Criteria	Description	Weight	Max pts
MVC Architecture	Correct MVC separation; no logic leakage; front controller pattern; router with parameter support.	25%	**25**
PSR-4 Autoloading	Valid composer.json with correct PSR-4 namespaces for Core and App; no manual require statements.	10%	**10**
SOLID Principles	Each principle evidenced in class design; supported by written justification with specific class references.	25%	**25**
MVP Application	Functional CRUD with at least 5 routes, form handling, input validation, and database interaction.	25%	**25**
Code Quality	PHP 8.2 features used appropriately; type safety; meaningful naming; no deprecation warnings.	10%	**10**
Documentation	Clear README with setup instructions, route list, and design decisions; inline code comments where necessary.	5%	**5**
Total		**100%**	**100**

Pass threshold: 60 points or higher. **Academic integrity:** Plagiarism or AI-generated framework code results in an automatic zero.

7. Notes & Constraints

> You may NOT use any existing PHP framework (Laravel, Symfony, CodeIgniter, Slim, etc.) as your framework base. Micro-libraries for specific tasks — such as a templating engine or Doctrine DBAL — are permitted if declared in composer.json and justified in your README.

> Your framework must run on PHP 8.3 or higher. Use of typed properties, match expressions, named arguments, constructor property promotion, and readonly properties is encouraged and demonstrates mastery of the language.

> All classes must be autoloaded via Composer PSR-4. No manual require or include statements for class files are permitted anywhere except public/index.php. Violations will result in a zero for the PSR-4 criterion.

Suggested MVP Ideas

Choose any of the following, or propose your own idea (subject to instructor approval):

- Blog platform — Posts, categories, comments, and a contact form.
- Task manager — Projects, tasks, due dates, and status tracking.
- URL shortener — Link creation, redirect handling, and click analytics.
- Contact book — CRUD for contacts with search and tagging.
- Inventory system — Products, stock levels, and basic reporting.

Good luck. Build something you are proud of.